KURZANLEITUNG

BIDA ML Starter

Stand **20.10.2026** Aufgaben **Forecasting · Klassifikation · Regression**

Umgebung **uv, conda oder WinPython, Ergebnisse nach {env}_ML**

IN EINER MINUTE

Das Projekt auf den Punkt

1 **Eine Vorlage für jedes ML-Projekt**
Forecasting, Klassifikation und Regression laufen nach demselben Muster: Daten prüfen, Baseline, Modelle, Evaluation, Ergebnis nach Snowflake.

2 **Anleitung statt fertigem Code**
Die Template-Notebooks beschreiben jeden Schritt und warum er nötig ist. Code enthalten sie keinen.

3 **Claude Code schreibt den Code**
Pro Schritt fragst du Claude Code. Die Skills liefern geprüften Referenzcode, den Claude an deine Daten anpasst.

4 **Leitplanken sind eingebaut**
Hooks und Rules schützen Zugangsdaten und Datenbanken. Was gefährlich ist, blockiert das Projekt selbst.

5 **Ergebnisse gehen in die ML-Datenbank**
Geschrieben wird nur nach `{env}_ML`: entwickelt in DEV, getestet in INT. Nach PROD schreibt erst das offizielle Deployment.

6 **Du entscheidest**
Claude schlägt vor, du prüfst den Code, führst ihn aus und beurteilst das Ergebnis.

WOZU

Sinn und Zweck

Ohne Vorlage baut jedes ML-Projekt seine eigene Struktur, eigene Verbindungen und eigene Regeln. Der BIDA ML Starter nimmt das vorweg. Du startest mit einer erprobten Methodik und einer fertigen Infrastruktur und konzentrierst dich auf deine Daten und dein Modell.

Einheitlich
Jedes Projekt ist gleich aufgebaut. Wer eines kennt, findet sich in allen zurecht.

Reproduzierbar
Feste Paketversionen, zentrale Config, `RANDOM_STATE = 42`. Ein Lauf lässt sich später genau nachstellen.

Methodisch sauber
Baseline zuerst, Zeitreihen zeitlich teilen, Klassifikation stratifiziert. Jedes Modell muss die Baseline schlagen.

Nachvollziehbar
Jeder Lauf schreibt `run_id`, Modellname, Version und Zeitstempel mit. Ergebnisse werden angehängt, nie überschrieben.

Sicher
Keine Zugangsdaten in Git, kein destruktives SQL, Schreiben nur in die ML-Datenbank der eigenen Umgebung.

Wiederverwendbar
Skills, Notebooks und `src/` übernimmt das Agentic Engineering Starter Template für neue Projekte.

IN EINER STUNDE STARTKLAR

So kommst du zu deinem ersten Modell

1 **Repository klonen**
`git clone https://github.com/themanbetterknownasthemachine/BIDA-ML-Starter.git`, dann den Ordner `BIDA-ML-Starter` in VS Code öffnen.

2 **Umgebung wählen**
Mit uv: `uv sync --extra cpu`, mit NVIDIA-GPU `--extra gpu`. Ohne Installationsrechte den vorbereiteten WinPython-Ordner als Interpreter wählen, siehe README.

3 **Zugangsdaten anlegen**
Erst wenn du mit Snowflake arbeitest: `.env.example` nach `.env` kopieren, Account, User und eine Auth-Methode eintragen.

4 **Smoke-Test ausführen**
Kernel `.venv` oder WinPython wählen und `00_environment_check.ipynb` ausführen. Ohne Snowflake darf die Snowflake-Zelle fehlschlagen.

5 **Arbeits-Notebook anlegen**
Zum Beispiel `notebooks/work_<projekt>.ipynb`. Daneben das passende Template öffnen.

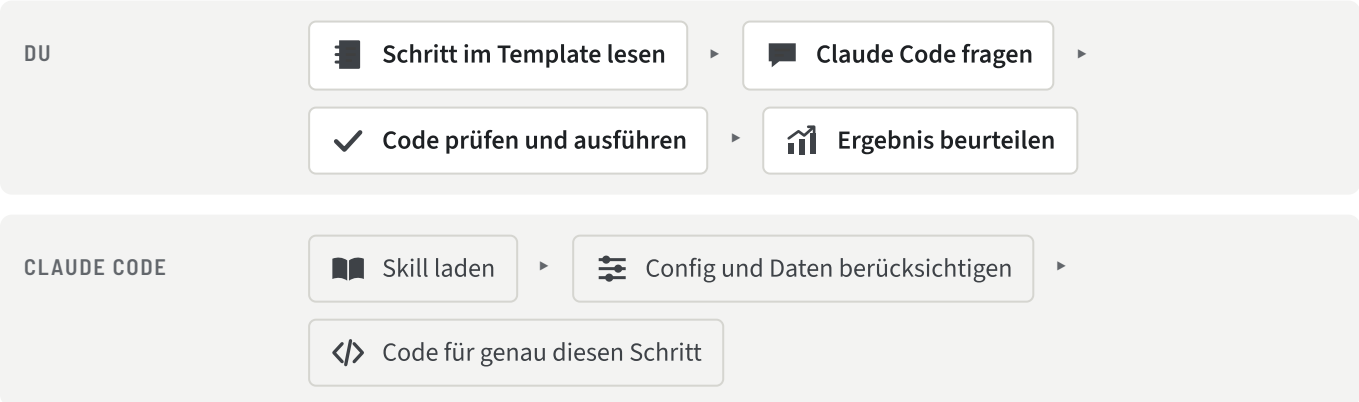
6 **Claude Code nach dem Code fragen**
Pro Schritt eine Frage, zum Beispiel: Schritt 8 aus 02_forecasting für meine Daten.

7 **Ergebnisse nach Snowflake schreiben**
`write_to_snowflake(df, "TABELLE")` schreibt in die ML-Datenbank deiner Umgebung, auf dem Laptop `DEV_ML`.

DAS PRINZIP

Anleitung lesen, Code von Claude, Urteil von dir

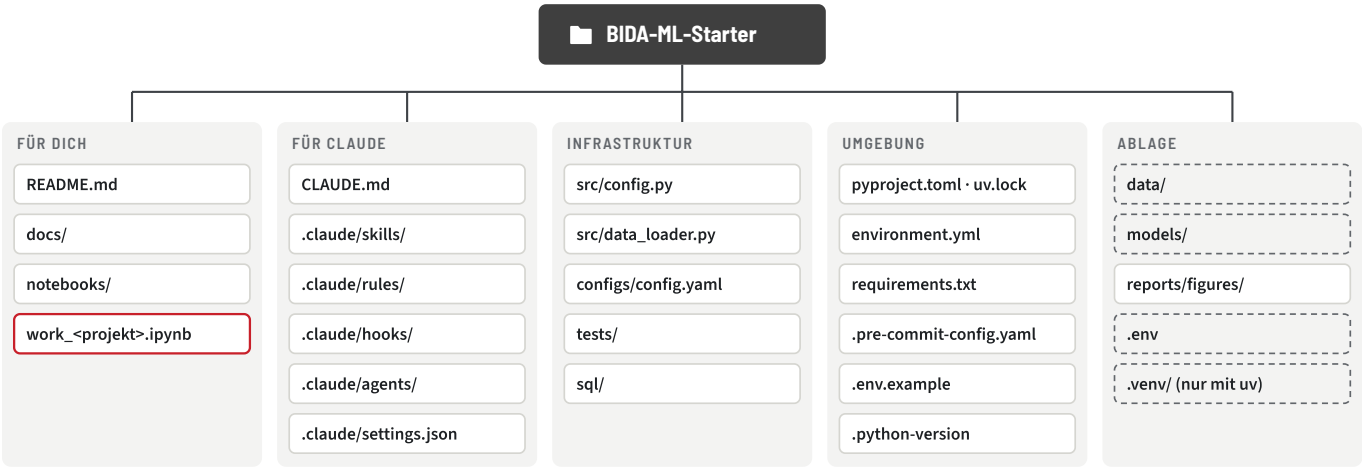
Die Template-Notebooks enthalten bewusst keinen Code, sondern beschreiben, was zu tun ist und warum. Den Code schreibt Claude Code auf Basis der Skills. Du prüfst ihn und entscheidest.



Der Code gehört in dein Arbeits-Notebook. Template-Notebooks und `src/` bleiben unverändert.

WO WAS IST

So ist das Projekt aufgebaut



bleibt lokal, nicht in Git legst du selbst an

WOFÜR WELCHER ORDNER

Die Ordner

notebooks/
Anleitungen

`00_environment_check` prüft die Umgebung und ist das einzige Notebook mit Code. `01` bis `04` sind Anleitungen für Datenexploration, Forecasting, Klassifikation und Regression.

Dein Code: in einem eigenen Arbeits-Notebook daneben, zum Beispiel `work_<projekt>.ipynb`. Die Templates bleiben unverändert.

---

src/
Infrastruktur

`config.py` lädt die Konfiguration und setzt die Umgebung ein. `data_loader.py` öffnet die Snowflake-Verbindung, liest Tabellen und schreibt Ergebnisse. Features, Modelle und Plots entstehen im Notebook, nicht hier.

Später: Funktionen, die mehrere Notebooks brauchen, dürfen nach `src/`, immer mit Test.

---

configs/
Konfiguration

`config.yaml` ist die zentrale Stelle für Projektname, Ziel-Datenbank, Schemas und Quelltabellen. Tabellennamen stehen hier, nicht im Code, und immer mit `{env}`.

---

tests/ und sql/
Qualität und Rechte

`tests/` enthält die Unit-Tests für `src/`, ausgeführt mit `python -m pytest`. `sql/` dokumentiert, welche Rechte die Rolle `ML_DEVELOPER` in Snowflake braucht. Das Skript ist eine Referenz für Admins, das Projekt führt es nicht aus.

---

data/, models/, reports/
Ablage

`data/raw/` für Rohdaten wie CSV, Excel oder Parquet, `data/processed/` für aufbereitete Daten. Trainierte Modelle unter `models/<name>_v<n>_<datum>/`. Plots und Metriken unter `reports/figures/`.

Nicht in Git: `data/` und `models/`. Daten und Modelle bleiben auf deinem Laptop.

README.md und docs/
Für das Team

Das README erklärt das Setup mit uv, conda oder WinPython, den Snowflake-Zugang und das Arbeiten mit dem Template. In `docs/` liegt diese Kurzanleitung als HTML und PDF.

Dateien im Hauptordner
Umgebung

`pyproject.toml` und `uv.lock` legen die Pakete und Versionen für uv fest, `environment.yml` und `requirements.txt` dieselben für conda. `.pre-commit-config.yaml` prüft vor jedem Commit den Code mit ruff und entfernt Notebook-Ausgaben. `.env.example` ist die Vorlage für deine Zugangsdaten in `.env`.

Mit WinPython: Die Pakete kommen aus dem WinPython-Ordner, diese Dateien gelten dann nicht.

WAS WO STEHT

Die Templates

00 Environment Check
Smoke-Test, mit Code

Prüft Python, Libraries, PyTorch mit GPU, die Snowflake-Verbindung und die Config. Nach jedem Setup einmal ausführen.

Erwartet: Rolle ML_DEVELOPER, Warehouse CONSUMER, Datenbank DEV_ML.

01 Datenexploration
EDA, 8 Schritte

Überblick, fehlende Werte, Verteilungen, Korrelationen, zeitliche Muster, Ausreisser. Am Ende die Empfehlung, welches Template folgt.

Skill: keiner nötig, Claude schreibt den Code direkt aus der Anleitung.

02 Forecasting
Zeitreihen, 20 Schritte

Von einfach zu komplex: Baseline, statistische Modelle (AutoARIMA, ETS, Theta, SARIMAX), klassisches ML (LightGBM, XGBoost über MLForecast), neurale Modelle (N-HiTS, TFT, TSMixerx, PatchTST). Dazu Cross-Validation, Fehleranalyse nach Wochentag, SHAP, Tuning mit Optuna.

Skill: forecasting. **Datenformat:** unique_id, ds, y. **Metriken:** MAE, RMSE, MAPE, sMAPE.

03 Klassifikation
11 Schritte

sklearn-Pipeline mit Preprocessing, LightGBM oder XGBoost, stratifizierte Cross-Validation, GridSearch, Evaluation mit Confusion Matrix, ROC- und PR-Kurve, Feature Importance mit SHAP.

Skill: classification. **Metriken:** Accuracy, F1, ROC-AUC, PR-AUC.

04 Regression
16 Schritte

Outlier-Analyse der Zielvariable, Pipeline, Baseline, Cross-Validation, Tuning mit Optuna, Actual gegen Predicted, Residuen-Analyse, SHAP.

Skill: regression. **Metriken:** RMSE, MAE, R^2, MAPE.

CLAUDE CODE

Die Claude-Dateien

Claude Code liest beim Arbeiten mehrere Dateien mit. Sie erklären Claude das Projekt, liefern den Code für die Schritte und legen fest, was verboten ist.

CLAUDE.md

Das Projektgedächtnis. Claude liest es bei jedem Start: Aufbau, Snowflake-Struktur, Konventionen und Arbeitsregeln.

skills/

Code-Referenz für forecasting, classification und regression. Claude lädt den passenden Skill selbst. Mit `/forecasting` erzwingst du ihn.

rules/

Regeln nach Thema. `security.md` gilt immer. `notebooks.md` und `python.md` gelten nur, wenn Claude in `notebooks/` oder `src/` arbeitet.

hooks/

Laufen automatisch vor jeder Aktion von Claude. `protect-files.sh` blockt das Lesen von `.env` und Schlüsseln sowie DROP, DELETE und TRUNCATE.

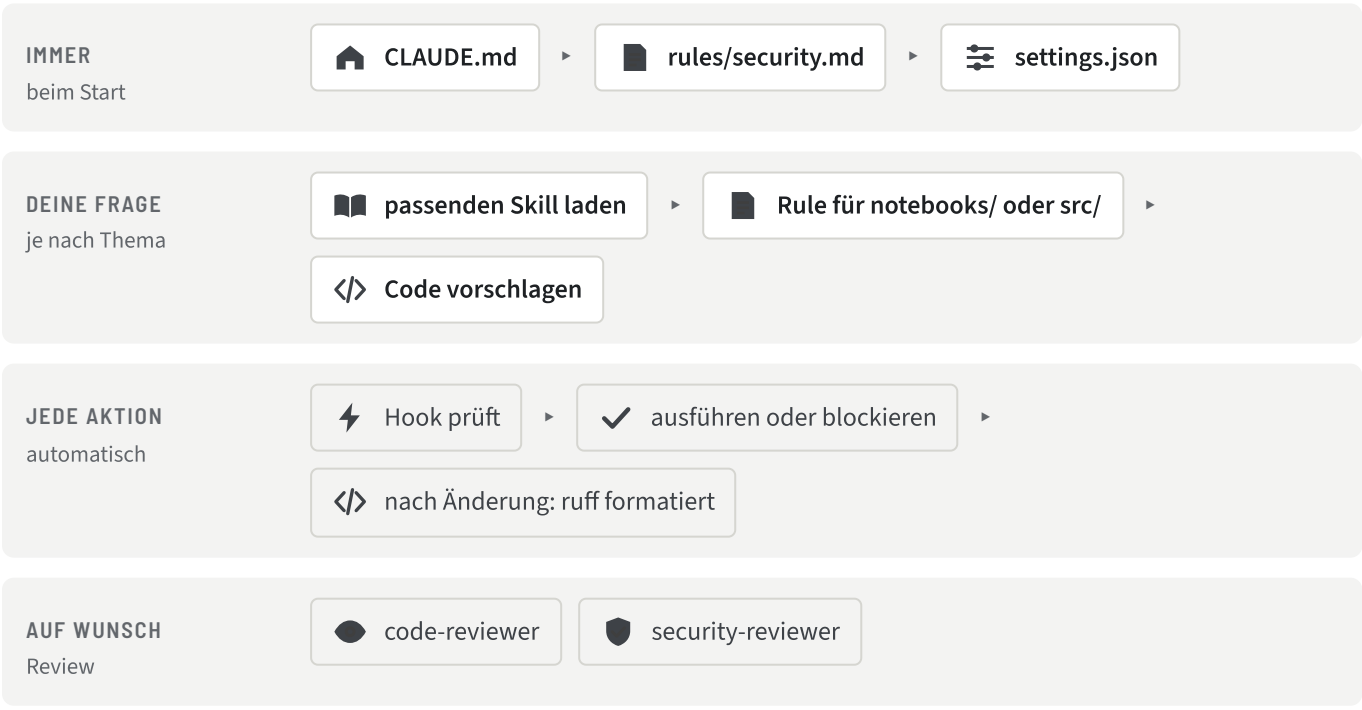
agents/

Zwei Prüfer, die nur lesen dürfen: `code-reviewer` für Korrektheit und Konventionen, `security-reviewer` für Secrets und unsicheres SQL.

settings.json

Team-Einstellungen in Git: was ohne Rückfrage läuft, was gesperrt ist, zum Beispiel `git push`, und welche Hooks aktiv sind. Persönliches gehört in `settings.local.json`, nicht in Git.

Was bei einer Frage an Claude passiert



Rules sind Anweisungen, Hooks sind Sperren. An CLAUDE.md und die Rules hält sich Claude, weil es sie liest. Die Hooks greifen technisch, egal was Claude entscheidet. Deshalb liegen die harten Verbote, also Zugangsdaten und destruktives SQL, im Hook.

DATEN

Der Weg der Daten

QUELLE
nur lesen

Snowflake `{env}_DATALAKE` oder Dateien in `data/raw/`

ARBEIT
auf deinem Laptop

Arbeits-Notebook ▸ `write_to_snowflake()`

ZIEL
{env}_ML

INFERENCE: Forecasts, Predictions

REGISTRY: Modell-Läufe

MONITORING: Metriken

UMGEBUNG
ML_ENV in .env

DEV_ML: entwickeln ▸ INT_ML: testen ▸

PROD_ML: nur offizielles Deployment

DANACH
DWH-Team

Rückfluss ins DWH ▸ Power BI

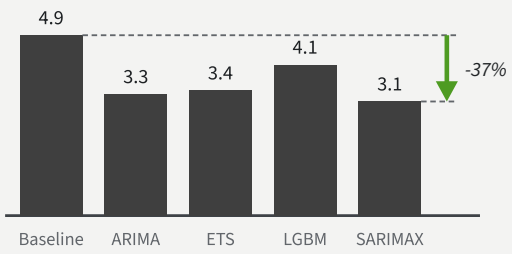
1 **Laden.** Aus Snowflake mit `load_query`, `load_table` oder `load_timeseries`. Dateien mit pandas, zum Beispiel `pd.read_parquet`.

2 **Schreiben.** Immer mit `write_to_snowflake`, immer anhängen statt überschreiben, immer mit `run_id`, Modellname, Version und Zeitstempel.

`ML_ENV` in `.env` ist der einzige Schalter. In Code und Config steht immer `{env}`, nie `DEV_` oder `PROD_`.

ERGEBNISSE

Modelle richtig vergleichen



Beispiel: MAE je Modell auf dem Test-Set. Tiefer ist besser. Der Pfeil vergleicht das beste Modell mit der Baseline.

Die Baseline ist Pflicht

Zuerst ein einfaches Referenzmodell, zum Beispiel Seasonal Naive oder DummyRegressor. Jedes weitere Modell muss sie schlagen.

Der Split entscheidet

Zeitreihen immer **zeitlich** teilen, nie zufällig. Klassifikation immer **stratifiziert**. Sonst sind die Metriken zu optimistisch.

Nur bekannte Features

Exogene Features müssen in der Zukunft bekannt sein: Kalender, Feiertage, geplante Aktionen. Lag-Features nie aus der Zukunft.

Eine Zahl reicht nicht

Cross-Validation zeigt die Streuung, Fehleranalyse und Residuen zeigen, *wo* das Modell danebenliegt.

Vorsicht bei MAPE mit Nullwerten. Enthält die Zielgrösse Nullen oder sehr kleine Werte, explodiert der MAPE oder ist nicht definiert. Dann sMAPE oder MAE verwenden. Das ist kein Modellfehler, sondern eine Eigenschaft der Kennzahl.

CLAUDE CODE

So fragst du richtig

Nenne das Template, die Schritt-Nummer und was an deinen Daten besonders ist.

Gut: Schritt 7 aus 02_forecasting. Meine Daten sind täglich von Montag bis Freitag, Horizont 10 Tage, drei Serien. Exogene Features: Feiertage Kanton Luzern.

Gut: Die Baseline hat MAE 4.9, LightGBM 4.1. Analysiere die Fehler nach Wochentag (Schritt 14) und schlage Features vor.

Zu vage: Mach mir ein Forecast-Modell.

Skill gezielt laden

`/forecasting`, `/classification`, `/regression`. Sonst wählt Claude den Skill selbst.

Zuerst klärt Claude

Datenquelle, Zielvariable und Horizont. Fehlt etwas in der Config, weist Claude darauf hin.

Was Claude nicht darf

`.env` und Schlüssel lesen, DROP, DELETE oder TRUNCATE ausführen, `git push`. Das erzwingen der Hook und `settings.json`.

HINWEISE

Gut zu wissen

> **Tools immer mit `python -m` starten.** Auf den Pistor-Notebooks sind Launcher-EXEs blockiert, zum Beispiel `pytest.exe` und `pre-commit.exe`. Deshalb `python -m pytest`, mit uv `uv run python -m pytest`.

Zwei Wege zur Umgebung

Mit uv entsteht eine `.venv` im Projekt. Auf Geschäftsnotebooks ohne Installationsrechte nutzt du den vorbereiteten WinPython-Ordner. Code und Notebooks sind in beiden Fällen dieselben.

Fehlt ein Paket in WinPython

Nicht selbst installieren, sondern beim WinPython-Verantwortlichen anfragen, damit alle denselben Stand haben. Bis dahin schlägt Claude Alternativen aus scikit-learn oder statsmodels vor.

GPU

Mit uv installiert `--extra gpu` PyTorch für CUDA 13.0, das braucht einen NVIDIA-Treiber ab Version 580. WinPython bringt PyTorch mit CUDA bereits mit. Der Smoke-Test zeigt `CUDA: True`.

Schlüssel und .env

Die `.env` enthält nur den Pfad. Der private Schlüssel liegt unter `~/.snowflake/rsa_key.p8` und verlässt den Laptop nie. Den öffentlichen Schlüssel hinterlegt der Snowflake-Admin. Bis dahin hilft `SF_AUTHENTICATOR=externalbrowser`.

Abhängigkeiten ändern

Nur in `pyproject.toml`. Danach `uv lock` und `uv export --no-hashes --group dev -o requirements.txt`, damit conda dieselben Versionen erhält. Beides committen.

Was nicht in Git ist

`.env`, Schlüssel, `data/`, `models/`, `.venv/` und `settings.local.json`. Dafür sorgt die `.gitignore`.

Ausgaben vor dem Commit leeren

Notebook-Ausgaben können Daten enthalten. Mit uv entfernt sie pre-commit automatisch. Mit WinPython vor dem Commit **Clear All Outputs** klicken.

Vom Experiment zum Projekt

Ein echtes Vorhaben mit Spec und Verifier startest du aus dem Agentic Engineering Starter Template. Es kopiert Skills, Notebooks, `src/` und `configs/` von hier, Notebook und Skill immer als Paar. Die Pfade deshalb nicht umbenennen.

ZUM NACHSCHLAGEN

Die wichtigsten Befehle

`uv sync --extra cpu`	Umgebung mit uv erstellen oder aktualisieren. Mit GPU: `--extra gpu`.
`conda env create -f environment.yml`	Dieselbe Umgebung mit conda, danach `conda activate bida-ml`.
`python -m pytest`	Tests ausführen. Mit WinPython in der WinPython Command Prompt, mit uv als `uv run python -m pytest`.
`python -m ruff check --fix src tests`	Python-Code prüfen und automatisch korrigieren. Mit uv: `uv run ruff check --fix src tests`.
`uv run python -m pre_commit install`	Einmalig mit uv: Git-Hooks für ruff und nbstripout aktivieren.
`git push`	Machst immer du selbst. Claude Code darf committen, aber nicht pushen.